AeonG: An Efficient Built-in Temporal Support in Graph Databases (Extended Version)

Jiamin Hou[†], Zhanhao Zhao[†], Zhouyu Wang[†], Wei Lu[†], Guodong Jin[§], Dong Wen[‡], Xiaoyong Du[†]
[†] *Renmin University of China* [§] *University of Waterloo* [‡] *University of New South Wales*
[†]{jiaminhou, zhanhaozhao, zyu_wang, lu-wei, duyong}@ruc.edu.cn
[§]g35jin@uwaterloo.ca [‡]dong.wen@unsw.edu.au

ABSTRACT

Real-world graphs are often dynamic and evolve over time. It is crucial for storing and querying a graph's evolution in graph databases. However, existing works either suffer from high storage overhead or lack efficient temporal query support, or both. In this paper, we propose AeonG, a new graph database with built-in temporal support. AeonG is based on a novel temporal graph model. To fit this model, we design a storage engine and a query engine. Our storage engine is hybrid, with one current storage to manage the most recent versions of graph objects, and another historical storage to manage the previous versions of graph objects. This separation makes the performance degradation of querying the most recent versions of graph objects as slight as possible. To reduce the historical storage overhead, we propose a novel *anchor+delta* strategy, in which we periodically create a complete version (namely anchor) of a graph object, and maintain every change (namely delta) between two adjacent anchors of the same object. To boost temporal query processing, we propose an anchor-based version retrieval technique in the query engine to skip unnecessary historical version traversals. Extensive experiments are conducted on both real and synthetic datasets. The results show that AeonG achieves up to 5.73× lower storage overhead, 2.82× lower latency for graph operations, and 2.57× lower latency for temporal queries against the next-best system, while introducing only 9.74% performance degradation for supporting temporal features.

PVLDB Reference Format:
Jiamin Hou[†], Zhanhao Zhao[†], Zhouyu Wang[†], Wei Lu[†], Guodong Jin[§], Dong Wen[‡], Xiaoyong Du[†] . AeonG: An Efficient Built-in Temporal Support in Graph Databases (Extended Version). PVLDB, 17(1): XXX-XXX, 2024.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/hououou/AeonG.git.

1 INTRODUCTION

Graphs are prevalent to model relationships between real-world entities. Many graph databases, such as Neo4j [1], ArangoDB [2], Dgraph [3], and Memgraph [4], are developed to manage graph data efficiently. Despite the fact that real-world graphs are often dynamic and evolve over time, these databases are typically designed to manage up-to-date graph data: when a graph changes, the database only stores the current (latest) state of the graph, i.e., the most recent values of vertices and edges, while discarding any previous (historical) state. However, time-evolving (temporal) graph data, which contains both the latest and historical states of a graph, is important in many applications, such as financial fraud detection [5], traffic prediction in road networks [6], etc.

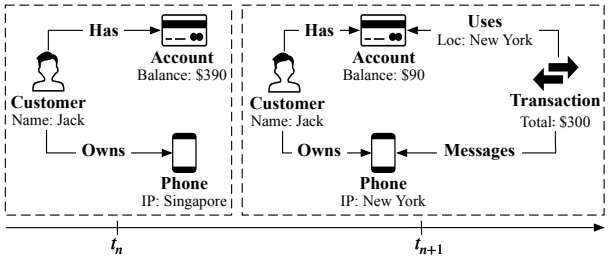


Figure 1: The Evolution of a Customer Purchase Graph

Example 1. Figure 1 shows the evolution of a customer purchase graph, where customers, bank accounts, phones, and transactions are modeled as entities, and the relationships between entities are modeled as edges. The phone logs its location during various activities, such as receiving a message or web browsing. Each customer purchase invokes a transaction to update the account balance and record the location where it occurs. Let us assume that at time t_n, a customer named Jack has an account balance of $390, and his phone's location falls in Singapore. We store a graph reflecting this state, as shown in the left portion of Figure 1. Subsequently, at time t_{n+1} (one minute after t_n), Jack invokes a purchase transaction totaling $300, resulting in a new graph state, as shown in the right part of Figure 1. This transaction occurs in New York, identical to the location of Jack's phone, thus it appears to be legitimate. However, when comparing the states of t_{n+1} and t_n, we observe that Jack's phone location changes from Singapore to New York within one minute. Considering it is impossible for Jack to travel such a distance so quickly, this transaction is likely fraudulent. We would like to emphasize that changes in phone location alone are not inherently suspicious. However, when such a location shift is associated with a transaction, it warrants vigilance. As discussed above, this potentially fraudulent activity can only be identified by tracking the evolution of the graph structure over time. Therefore, traditional graph databases, which only maintain the latest state at t_{n+1}, would fail to detect such fraudulent transactions. □

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

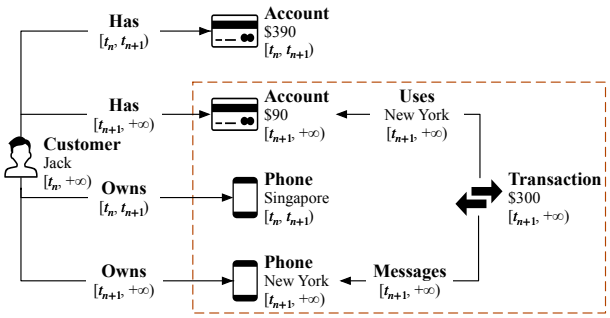


Figure 2: Customer Purchase Graph with a Time Dimension

Thus far, various works have been proposed to manage temporal graph data. Several proposals [7–13] assign each vertex or edge in the graph with a timestamp property to reflect its lifespan, as shown in Figure 2. Rather than discarding the previous state when the graph changes, these approaches maintain both the current and historical states in a single graph. For example, at time t_{n+1}, two vertices of Jack's phone coexist – one represents the previous state with a time interval of $[t_n, t_{n+1})$, and the other denotes the current state with $[t_{n+1}, +\infty)$. Consequently, they can detect fraudulent transactions, as in Example 1, by identifying the irregular sub-graph structure (highlighted in the red box) that indicates a transaction proceeded with location inconsistency. However, in these approaches where timestamps are treated as regular properties, executing temporal queries (which select data based on given timestamps) often requires the traversal of the entire graph. As the graph size inevitably increases due to the addition of historical states, query efficiency can significantly degrade over time. Another line of research [14–23] manages time-evolving graph data by periodically materializing the snapshots of the entire graph and logging the deltas between two successive snapshots. Querying a historical state in these methods requires first identifying the nearest snapshot based on the timestamp, and then reconstructing the complete graph state using the snapshot and associated deltas. Therefore, these methods incur substantial storage overhead due to the maintenance of complete snapshots, and can lead to sub-optimal query performance because of the historical state reconstruction.

Our goal is to design a graph database for efficient temporal graph data management. However, achieving this requires addressing three key challenges to minimize storage overhead and facilitate swift temporal query processing. First, as the volume of historical graph states continuously escalates, achieving minimal storage overhead is not straightforward❶. Second, given the considerable amount of historical states, it is not trivial to process temporal queries efficiently while upholding data consistency ❷. Lastly, the database needs native temporal support to enable users to conveniently access temporal graph data❸.

In this paper, we propose AeonG, a new graph database that efficiently offers built-in temporal support. By integrating the widely-accepted static property graph model [24–27] with a time dimension, we first define a *temporal property graph model* to formalize the representation and manipulation of temporal graph data. Guided by this model, we then extend the common graph database architecture to design AeonG. In particular, we enhance the storage engine, query language, and query engine, with efficient temporal support.

We build a *hybrid storage engine*❶, constituting the current storage and historical storage, to store temporal graph data with minimal storage overhead. This engine maintains multiple versions for each vertex and edge, with the most recent versions retained in the current storage and previous versions in the historical storage. We integrate time dimensions into the data layout, and develop the current storage based on the multi-version storage engine used in various existing graph databases [2–4, 28–30]. We propose a novel "anchor+delta" strategy to compactly organize historical data in the historical storage. In particular, we periodically create a complete version (namely anchor) of a graph object and maintain every change (namely delta) between two consecutive anchors of the same object to reduce the historical storage overhead. Moreover, we introduce an asynchronous migration mechanism to transfer outdated versions from the current storage to the historical storage. Instead of synchronously migrating previous versions with every update or deletion of a vertex/edge, we defer the migration until the database's periodic garbage collection is invoked [31]. This mechanism ensures that the migration is non-intrusive, thereby reducing the performance degradation caused by the maintenance of temporal data.

We then present a *temporal-enhanced query language*❸, which extends Cypher [25], a common-used graph query language, to conveniently access temporal graph data. Building upon the hybrid storage engine, we introduce a built-in *temporal query engine*❷. We inherit two fundamental operations from existing graph databases, namely scan and expand, and extend them to enable consistent and efficient temporal query processing. We propose a unique anchor-based version retrieval technique to minimize unnecessary historical version traversals in the scan and expand operators. Specifically, we directly locate the nearest anchor that aligns with the given query conditions, and apply the subsequent deltas on the obtained anchors to reconstruct the desired version, thus minimizing the historical version traversal overhead.

In summary, we make the following contributions:

- We present AeonG, a new graph database providing efficient built-in temporal support. Built with a temporal-enhanced query language, query engine, and storage engine, AeonG regards temporal features as the first citizen, making it simple and intuitive to manipulate temporal graph data.
- We propose a hybrid storage engine, which employs separate storage engines with an "anchor+delta" strategy to reduce storage overhead for historical data. We further introduce an asynchronous migration strategy to minimize performance degradation for maintaining temporal graph data.
- We design a temporal query engine, featuring an anchor-based version retrieval technique, to provide consistent and efficient temporal query processing with minimal historical version traversal overhead.
- We implement AeonG based on Memgraph [4], a real-world native graph database. We conduct extensive experiments on both real and synthetic datasets, and compare AeonG against two state-of-the-art temporal graph databases [7, 19]. The results demonstrate that AeonG achieves up to 5.73× lower storage overhead, 2.82× lower latency for graph operations, and 2.57×

lower latency for temporal queries, while only inducing 9.74% performance degradation for supporting temporal features.

The remainder of the paper is structured as follows. The next section formulates the temporal graph model, and presents the temporal query language. Section 3 overviews the architecture of AeonG. Section 4 details the hybrid storage engine, and Section 5 elaborates on the temporal query engine. Section 6 describes the system implementation. Section 7 presents the experimental results. Section 8 discusses the related work, and Section 9 concludes.

2 MODELING AND QUERY LANGUAGE

In this section, we formulate the temporal graph model and present the temporal query language used in AeonG.

2.1 Temporal Property Graph Model

We define the temporal graph model by extending the static property graph model [24–27] with a time dimension. In the property graph model, real-world entities are represented as vertices, and the relationships between these entities are modeled as edges. Each vertex or edge has a unique identifier (id for short), possibly several labels (e.g., customer, phone), and properties (e.g., Name: Jack).

Definition 1 (Property Graph). Let $\mathcal{N}$ and $\mathcal{E}$ denote sets of vertex ids and edge ids, respectively. Assume countable sets $\mathcal{L}$, $\mathcal{K}$, and $\mathcal{V}$ of *labels*, *property names*, and *property values*. A property graph is a tuple $G = \langle N, E, \rho, \lambda, \pi \rangle$ where:

- N is a finite subset of $\mathcal{N}$, whose elements are referred to as the *vertices* of G;
- E is a finite subset of $\mathcal{E}$, whose elements are referred to as the *edges* of G and $N \cap E = \emptyset$;
- ρ: $E \rightarrow (N \times N)$ is a total function mapping each edge to its source and destination vertices;
- $\lambda : (N \cup E) \rightarrow 2^{\mathcal{L}}$ is a total function mapping vertices and edges to finite sets of labels (including the empty set);
- $\pi : (N \cup E) \times \mathcal{K} \rightarrow \mathcal{V}$ is a finite partial function, mapping a vertex/edge and a property key to a value.

The property graph model, originally designed for static graphs, lacks the inherent ability to capture the evolution of graphs over time. In the context of relational databases, the concept of "Transaction Time" [32] is proposed to bring a time dimension to the relational model. This transaction time is created and maintained by the database system itself, tracking the lifespan of each data item within the system. Inspired by the transaction time, we integrate the time dimension into the property graph model to formally define the temporal property graph model.

Definition 2 (Temporal Property Graph). A temporal property graph is a tuple $G = \langle \Omega, N, E, \rho, \lambda, \pi, \sigma, \tau \rangle$ where:

- Ω is a temporal domain, which is a finite set of consecutive timestamps, that is, $\Omega = \{i \in T | a \leq i \leq b\}$ for some $a, b \in T$ such that $a \leq b$. T represents the universe of time points;
- N, E, ρ, λ, π inherit their definitions from Definition 1;
- $\sigma : (N \cup E) \times \Omega \rightarrow \{true, false\}$ is a total function that maps a vertex or an edge and a time period ω to a Boolean variable, indicating whether this vertex or edge exists during period ω;

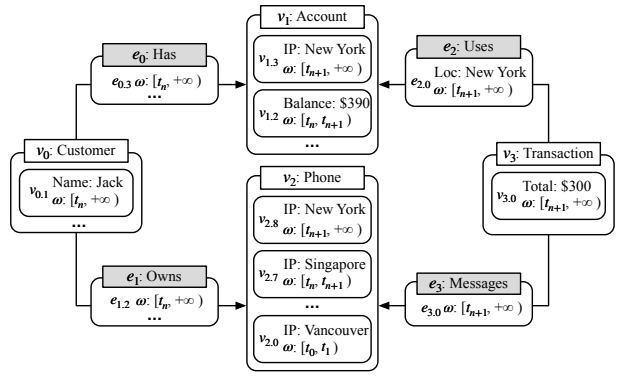


Figure 3: Example of a Temporal Property Graph

- $\tau : (N \cup E) \times \mathcal{K} \times \Omega \rightarrow \mathcal{V}$ is a partial function that maps a vertex or an edge and a property key, and a time period ω to a value.

Constraints. In our temporal property graph model, we impose two constraints to enforce that the graph at any time point corresponds to a valid property graph. At any time point t: (1) An edge exists only if both source and destination vertices exist at t. Formally, if $e \in E$, $\sigma(e, t) = true$, and $\rho(e) = (v_1, v_2)$, then $\sigma(v_1, t) = true$ and $\sigma(v_2, t) = true$; (2) A property can only take on a value during the time period when the corresponding vertex or edge exists. Formally, if $\tau(o, k, t) = val$, where $o \in (N \cup E)$, $k \in \mathcal{K}$, $val \in \mathcal{V}$, then $\sigma(o, t) = true$.

In our model, each graph object comprises multiple corresponding versions, including one current/latest version and potentially several historical versions. Unlike existing works such as T-GQL, which assigns a time period to each graph object, our model assigns the time period to each version of a graph object (vertex or edge). For example, as depicted in Figure 2, consider updating the entity "Phone". In existing models, this update results in retaining two entire "Phone" vertices within the same graph, leading to two redundant unchanged "Owns" edges. In contrast, we create a new version of the "Phone" vertex and re-link the "Owns" edge to this version, with changed attributes stored in the historical version. Consequently, our model is less complex but more efficient by avoiding the creation of redundant vertices and edges. At any given time t, a graph object version with $\omega = [st, ed)$ is said to be **legal** if $st \leq t < ed$. We classify a graph object version as a current version if it is legal at the current time, and as a historical version if it is not legal at the current time.

Graph operations. Our temporal model supports diverse graph operations as follows. Assume these graph operations are issued by a transaction committed at time t_1.

- *Creating* a vertex or an edge: This involves adding a vertex or edge with a current version having a time period $\omega = [t_1, +\infty)$.
- *Deleting* a vertex or an edge whose current version is with $\omega = [st, +\infty)$: This entails updating ω to $[st, t_1)$.
- *Updating* a vertex or an edge whose current version is with $\omega = [st, +\infty)$: This marks the current version as a historical version by updating ω to $[st, t_1)$ and generates a new current version with $\omega = [t_1, +\infty)$ representing the up-to-date semantics.

Example 2. According to Definition 2, we present the corresponding temporal property graph of Example 1 in Figure 3. Here,

Listing 1: Syntax of expressions, queries, and clauses in Cypher enhanced with temporal features

```
//EXPRESSIONS
expr::=v|a| f(expr_list) v ∈ 𝒱, a ∈ 𝒜, f ∈ ℱ                                                    values/variables
       |expr.k| {} | {prop_list}                                                                        maps
       |[] | [expr_list] | expr IN expr | expr[expr] | expr[expr..] | expr[..expr] | expr[expr..expr]     lists
       |expr START WITH expr | expr ENDS WITH expr | expr CONTAINS expr                               strings
       |expr OR expr| expr AND expr| expr XOR expr| NOT expr | expr IS NULL | expr IS NOT NULL          logic
       |expr < expr| expr <= expr| expr >= expr| expr > expr | expr = expr | expr <> expr          inequalities
expr_list::=expr | expr, expr_list                                                              expression lists

//QUERIES
query ::=query° | query UNION query | query UNION ALL query                                            unoins
query°::=RETURN ret | clause query°                                                        sequences of clauses
ret::= * | expr [AS a] | |ret, expr [AS a]                                                        return lists

//CLAUSES
clause::= [OPTIONAL] MATCH pattern_tuple [WHERE expr] [FOR TT AS OF expr | FOR TT FROM expr TO expr]   matching clauses
          |WITH ret [WHERE expr] | UNWIND expr AS a  a ∈ 𝒜                                        relational clauses
pattern_tuple::= pattern | pattern, pattern_tuple                                                 tuples of patterns
```

$\Omega = [t_0, +\infty]$, $N = [v_0, v_1, v_2, v_3]$, $E = [e_0, e_1, e_2, e_3]$. Each vertex and edge owns a current version and several historical versions from t_0 to t_n. For brevity, we omit graph states before t_n. At t_n, there exists three vertices (v_0, v_1 and v_2) and two edges ($e_0 : (v_0, v_1)$) and $e_1 : (v_0, v_2)$). For instance, v_2 owns a current version $v_{2.7}$, which has a unique id 2, a label "Phone", a property with the key-value pair (IP, Singapore), and a lifespan $\omega = [t_n, \infty)$. Subsequently, at t_{n+1}, consider there is a customer purchase transaction. It updates the properties of v_1 and v_2, resulting in new versions for each of them. Take v_2 as an example: it marks $v_{2.7}$ as a historical version and generates a new current version $v_{2.8}$. Specifically, τ maps $v_{2.8} \times IP \times [t_{n+1}, +\infty)$ to *New York* and maps $v_{2.7} \times IP \times [t_n, t_{n+1})$ to *Singapore*. We regard $v_{2.7}$ as legal at t_n, but not legal at t_{n+1}. Moreover, this transaction also creates v_3, e_2 and e_3, which have only the current version with $\omega = [t_{n+1}, +\infty)$. All the aforementioned graph operations adhere to the defined constraints. For instance, e_2 can be successfully created at t_{n+1} only after verifying linked vertices v_3 and v_1 exit at t_{n+1} (Constraint 1). □

2.2 Temporal Graph Query Language

AeonG incorporates a temporal-enhanced Cypher [25], which extends the standard syntax defined in OpenCypher [25] to support temporal queries. As illustrated in Listing 1, AeonG introduces two temporal syntax extensions in the `MATCH` clause (underlined in Listing 1): (1) FOR TT AS OF t, which retrieves all graph objects legal at time t, and (2) FOR TT FROM t_1 TO t_2, which locates all graph objects consistently legal within the time range from t_1 to t_2. The former is referred to as "time-point" queries, while the latter is known as "time-slice" queries. Users can apply any time conditions to temporal queries, spanning a wide time range from the oldest historical records up to the most recent updates. Further, apart from retrieving temporal graph data of user interest using temporal queries, AeonG allows users to submit common (non-temporal) queries and data manipulation operations (creating, updating, and deleting) with the standard Cypher syntax.

Example 3. Consider the query "What was Jack's phone IP at t_n". This query can be answered by issuing the following statement, where the temporal syntax is underlined: "MATCH (:Customer name: 'Jack')-[r]-(p:Phone) FOR TT AS OF t_n return p.IP. "

3 SYSTEM ARCHITECTURE

In this section, we introduce the system architecture of AeonG as shown in Figure 4. AeonG includes a transaction manager which enables handling a sequence of graph operations with ACID properties. We process transactions by employing the Multi-Version Concurrency Control (MVCC) [33]. MVCC ensures that transactions only see a consistent snapshot of the data that is **visible** to them, thus enabling multiple transactions to work concurrently without interfering with one another [33–37]. Given our primary focus on temporal data management, we now describe how we utilize the MVCC mechanism to manage temporal data effectively. AeonG supports built-in temporal features through two major components: the storage engine and the query engine.

3.1 Storage Engine

The storage engine of AeonG has two physically isolated storages: current storage and historical storage. The current storage typically maintains the current versions of graph objects. In contrast, the historical storage manages historical versions of graph objects, which are asynchronously migrated from the current storage.

Current storage. As discussed in Section 2.1, graph involves under various graph operations. To efficiently record these changes, AeonG builds its current storage as a multi-version storage, maintaining multiple versions for each graph object. Each graph object includes one current version retaining the up-to-date state and is linked to a list of historical versions preserving the previous states. When a graph object is updated, instead of directly overwriting the data, we create a new current version and move the previous one to the list of historical versions. We further integrate time dimensions into the data layout and modification paradigm to trace accurate graph evolution. We will introduce the details in Section 4.1.

Historical storage. AeonG does not store historical versions in the current storage permanently. Instead, we migrate them to the historical storage for long-term maintenance. To handle the potentially large volume of historical data, we properly compress the historical storage. We organize migrated historical versions in a key-value format. The key contains the metadata of a historical version, including vertex/edge id and version's lifespan ω, while

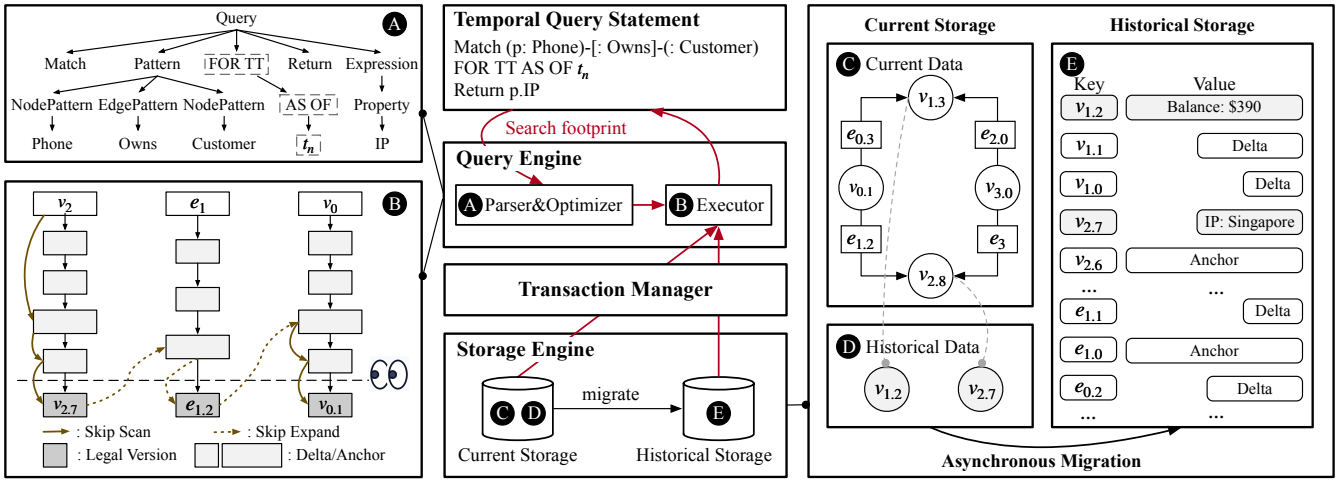


Figure 4: An Overview of AeonG - AeonG consists of a temporal query engine, a hybrid storage engine, and an MVCC-based transaction manager. We employ an "anchor+delta" strategy to reduce the historical storage overhead, while using an anchor-based version retrieval technique to ensure efficient temporal query processing.

the value holds detailed properties of this version. Instead of retaining all properties for every version, we organize versions in an "anchor+delta" manner. We utilize deltas to record relative differences between subsequent versions, minimizing the storage cost of ever-growing historical data. In addition, after a series of deltas, we maintain an anchor to preserve the complete state of a graph object, facilitating the reconstruction process when executing temporal queries. We will introduce the details in Section 4.2.

Asynchronous migration. AeonG utilizes an asynchronous migration approach to transfer historical data from the current storage to the historical storage. Rather than triggering a migration immediately following an update, this migration is postponed and occurs during the garbage collection of MVCC. This design ensures that transferring ever-growing historical data is lightweight, minimizing its overhead on the current storage. We will present our asynchronous migration in Section 4.2.

Example 4. In the right part of Figure 4, we demonstrate how AeonG stores the customer purchase graph as presented in Example 2. In the current storage, component Ⓒ records the current versions at t_{n+1}. Besides, the historical versions at t_n ($v_{1.2}$ and $v_{2.7}$ in this case), are stored in component Ⓓ . Take v_2 as an example. To capture the change in v_2's IP from Singapore to New York at t_{n+1}, AeonG performs two steps. First, it updates v_2 in place to create a new current version $v_{2.8}$. Second, to maintain the previous state, AeonG generates a historical version $v_{2.7}$, which is linked to $v_{2.8}$ in a chain and managed by MVCC. We migrate historical data in component Ⓓ to the historical storage (component Ⓔ) asynchronously. In the historical storage, the historical versions, $v_{1.2}$ and $v_{2.7}$, are organized as an anchor (represented as a long rectangle) and a delta (represented as a short rectangle). □

3.2 Query Engine

The query engine is responsible for handling user-issued queries, retrieving relevant graph data from the hybrid storage engine. Adhering to the "textbook" separation of components, AeonG consists of a parser, an optimizer, and an executor. While inheriting those components from existing graph databases, AeonG further extends them to support temporal queries.

Parser and optimizer. The parser translates queries and generates the corresponding syntax tree for the query optimizer. To accommodate the syntax of temporal queries, AeonG extends its lexical, syntactic, and semantic analyses to recognize time qualifiers as defined in Section 2.2. Leveraging the resulting syntax tree, the optimizer generates the execution plan for the executor.

Executor. AeonG builds upon and extends two core fundamental operations from traditional graph databases: scan and expand. The scan operator retrieves the required vertex versions for each query, while the expand operator fetches relevant edge and adjacent vertex versions. We enhance these operators to provide the consistent and efficient processing of temporal queries. To ensure consistent query results, we obtain current and historical data separately from two storage engines and then combine the results together. For current data, we follow the conventional query mechanism, which simply executes the plan over the current storage and captures visible graph object versions under MVCC's snapshot visibility check [33]. However, accessing historical data solely from the historical storage may yield incomplete results. Due to asynchronous migration, a portion of data is still in the current storage. To address this, we introduce a legal check mechanism that retrieves relevant data from both storages. This mechanism verifies if a version is legal within the given time condition to extract appropriate versions. Note that the snapshot visibility check is required when retrieving historical versions in the current storage. These steps ensure that the requested version(s) is from the consistent snapshot(s), thereby guaranteeing consistency.

To efficiently traverse historical versions from substantial historical data, we propose an anchor-based version retrieval technique to minimize unnecessary traversals. For the scan operator, we fetch relevant vertex versions that satisfy the provided temporal condition. To reconstruct a desired version, we directly locate the nearest anchor with a lifespan ω aligning with the query time constraint.

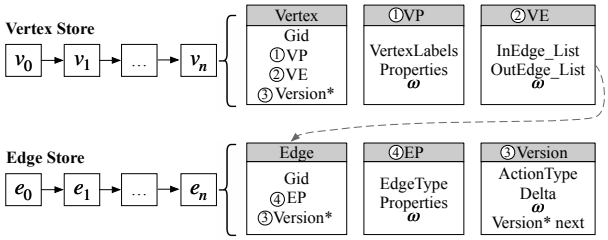


Figure 5: Data Layout of Current Storage

Subsequently, we traverse subsequent deltas from the obtained anchor, applying all fitting deltas. Regarding the expand operator, we further eliminate unnecessary traversals by directly locating the corresponding edge and adjacent vertex anchors using acquired held vertex versions. Further elaboration can be found in Section 5.

Example 5. Figure 4 Ⓐ depicts a simplified syntax tree for a given temporal query statement. Based on it, AeonG then utilizes the executor to fetch query results from the hybrid storage engine. Figure 4 Ⓑ illustrates the search footprint of the given query statement to answer "What were the phone IPs of all customers at t_n". We only reconstruct four relevant graph vertices/edges. We start to scan the vertex v_2, which we are interested in. We skip to seek its nearest anchor and collect all relevant deltas, to reconstruct the legal version $v_{2.7}$ we want. We then expand $v_{2.7}$ to get its linked edge $e_{1.2}$ and adjacency vertex $v_{0.1}$ without traversing the entire version chain of e_1 and v_0. □

4 HYBRID STORAGE ENGINE

In this section, we now elaborate on the design of AeonG 's hybrid storage engine.

4.1 Current Storage

Inheriting existing native graph databases [38], AeonG organizes graph data into three storage components: (i) vertex properties (VP), (ii) edge properties (EP), and (iii) graph topology, i.e., vertex's incoming and outgoing edges (VE). Like most native graph databases [1, 4, 39], we retain the topology within the vertices, enabling swift neighborhood traversal for each vertex. However, it is not trivial to record graph evolution under this design. The graph could change in not only its semantics, i.e., properties of graph objects, but also its structure. We have to identify different types of operations applied on the graph. For example, we should prevent the creation of a new vertex version when the vertex's relevant graph topology changes but its properties remain unchanged. To address this problem, we associate the time dimension to each independent storage component to separately record semantic changes and structural changes.

Data layout. As shown in Figure 5, the data store comprises two components: the Vertex Store, which maintains a list of vertex objects, and the Edge Store, which stores a list of edge objects. Every vertex object v has a unique graph identifier Gid, a VP part, a VE part, and a pointer to a linked list of historical versions (version chain). While every edge object e has a unique identifier Gid, an EP part, and a pointer to a linked list of historical versions. Specifically,

- The VP part stores a set of vertex labels and property value pairs associated with the current version of v, along with a time period ω indicating v's current semantic lifespan.
- The VE part keeps track of the current version of v's incoming and outgoing edges, with each entry in a list of (edge Gid, neighbor vertex Gid) pairs. The VE part also includes ω to record v's current structural lifespan.
- The EP part stores an edge type and property value pairs of the current version of e, along with a time period ω indicating e's current semantic lifespan.
- Each historical version contains: an action type indicating the changes made to a VP part, VE part, or EP part, a delta recording the steps to revert the changes to restore the previous version, a time period ω capturing the lifespan of the historical version, and a pointer to the next historical version. All historical versions generated by the same transaction are clustered in an undo buffer following the MVCC mechanism.

Modification paradigm. We now discuss how AeonG evolves the graph to handle various graph operations. Suppose the graph operation is invoked by a transaction T_i whose commit time is t_i. The modification paradigm on the graph data layout is as follows.

(1) When a vertex is created, we create a vertex object, set its VP part's time as $\omega = [t_i, +\infty)$, set its VE part's time as $\omega = [-\infty, +\infty)$, and link it in the vertex object lists.

(2) When an edge is created, we create an edge object, set its EP part's time as $\omega = [t_i, +\infty)$, and link it in the edge object lists. We also create connections to relevant vertex objects by setting their VE part's time to $\omega = [t_i, +\infty)$ if their previous VE part's time is $[-\infty, +\infty)$.

(3) When updating/creating/deleting a property value of a vertex object with $\omega = [t_j, +\infty)$, we first update relevant property values in the VP part and set its time as $\omega = [t_i, +\infty)$. Next, we create a historical VP version capturing the state of the vertex prior to the modification and set its ω as $[t_j, t_i)$. This VP version is then linked to the vertex's version chain. Updating a property value of an edge follows the same logic.

(4) When a vertex is deleted, we first delete all property values of the relevant vertex object, following the (3) paradigm, and then delete all connected edges, following the (5) paradigm.

(5) When an edge is deleted, we decompose it into the deletion of all property values and the deletion of connections with relevant vertices. The former acts on the edge object, following the (3) paradigm. The latter acts on the source and destination vertex object. Take the source vertex object with $\omega = [t_j, +\infty)$ as an example. We first update its VE part to delete this edge from outgoing edge lists and set its VE part's time as $\omega = [t_i, +\infty)$. We then create a VE version to record this deleted edge with $\omega = [t_j, t_i)$ and link it to the edge's version chain.

Example 6. We illustrate the data layout of current storage. As shown in Figure 6, we reconsider Example 2. To further represent the structural change, we suppose an event deleting e_2 at t_{n+2}. We focus on two vertices v_1 and v_3, and two edges e_0 and e_2 to showcase the graph evolution. At t_{n+1}, the transaction T_1, representing a customer purchase, is committed. It updates the VP part of v_1 and v_2, generates two VP versions linked to them, and creates graph

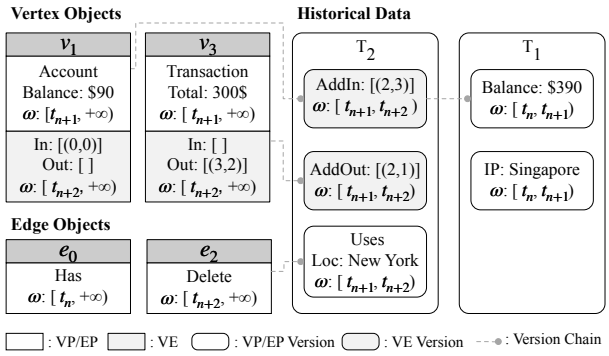


Figure 6: Layout Example of Current Storage

objects e_2, e_3, and v_3. Figure 6 shows these elements except e_3 and v_2 due to space limitations. At t_{n+2}, T_2 is committed to delete e_2, which affects v_1, v_3, and e_2 objects. It first acts on e_2's EP part to clear all semantic information and generates an EP version to record the previous edge state. Then, it acts on the VE part of v_1 and v_3 and generates two VE versions. □

4.2 Historical Storage

In MVCC, historical versions are not retained in the current storage permanently. Instead, once these versions are no longer needed by any active transaction, they are safely removed through garbage collection (GC) to optimize the performance of the current storage. AeonG utilizes this mechanism to transfer those inaccessible versions to the historical storage for long-term maintenance. For the sake of communication, historical versions in the current storage are referred to as "unreclaimed", while those in the historical storage are referred to as "reclaimed". In this subsection, we first present the optimized key-value format used for storing historical versions and then outline the process of migrating unreclaimed versions into the historical storage.

KV format. Reclaimed versions are organized in a key-value format, where the key represents the metadata of the version and the value stores the corresponding detailed information. AeonG groups three types of historical versions (VP, EP, and VE versions) into their respective segments. In each segment, the key is formed by combining three elements: the *Type* prefix, the graph identifier *Gid*, and ω of the version. The *Type* prefix indicates the type of version contained in the segment: 'V' for VP versions, 'E' for EP versions, and 'VE' for VE versions. The *Gid* is a unique identifier of the graph object linked to the version, while ω represents the version's lifespan. As for the value field, it contains the remaining semantic information, i.e., the delta of the version recording only data changes compared to the previous version. By organizing the data in this way, data sharing the same prefix in the key are physically clustered together in a SkipList [40], which ensures different versions of the same entity are automatically sorted based on their lifespan ω. As a result, it becomes efficient to retrieve in chronological order. Figure 7 depicts the reclaimed historical versions' KV format of unreclaimed historical versions in Figure 6.

Anchor+delta. We utilize deltas to reduce the storage overhead. However, retrieving a reclaimed graph object requires assembling the latest version with all previous deltas, incurring significant reconstruction costs for long retrieval histories. To mitigate this, we

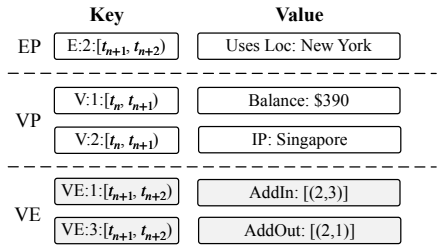


Figure 7: Key-value Format in Historical Storage

introduce anchors at intervals in the delta data, where an anchor represents the complete state of a graph object. To differentiate anchors from deltas in the KV store, we append a one-bit character suffix to the key's *Type*, where 'A' denotes anchors and 'D' represents deltas. Specifically, to reconstruct a certain reclaimed version o_1, we seek its most recent anchor o_2, collect all deltas from o_2 to o_1, and combine them to reconstruct o_1.

We propose an adaptive anchoring approach, which assigns different anchor intervals u for different graph objects. A higher u leads to more deltas between successive anchors, potentially increasing query latency but reducing storage overhead. Therefore, we assign a larger u to frequently updated objects to strike a balance between query latency and storage efficiency. Given a graph object o, we use Equation 1 to determine its u according to the update frequency $f(o)$, the number of updates conducted on o. This equation categorizes update frequencies into three levels (low, medium, and high) using two thresholds (τ_1 and τ_2). Each frequency level is assigned a specific anchor interval, calculated heuristically by multiplying the respective threshold values with a predefined parameter c. Currently, AeonG enables users to set parameters in Equation 1, such as τ_1, during database initialization and runtime. We acknowledge the potential of using deep learning methods, such as reinforcement learning, to automatically optimize these parameters based on specific workload requirements. However, since it is orthogonal to the focus of this work and considering the constraints of space, we leave this optimization in our future work.

$$u_o = \begin{cases} \tau_1 * c & f(o) \leq \tau_1 & \text{low frequency} \\ \tau_2 * c & \tau_1 < f(o) \leq \tau_2 & \text{medium frequency} \\ \tau_2^2/\tau_1 * c & \tau_2 \leq f(o) & \text{high frequency} \end{cases} \quad (1)$$

Data migration. In MVCC, unreclaimed historical versions will be physically removed from the current storage through an asynchronous GC phase when their relevant commit transactions are no longer active. AeonG collects those versions and migrates them to the historical storage for long-term maintenance, as detailed in Algorithm 1. We use *undo* to maintain the unreclaimed version to be migrated (line 2) and *kv* to store reclaimed data in a key-value format (line 3). Each unreclaimed version *undo* in the *CT* is initially encoded into a key-value pair *kv* (line 5). Subsequently, we store it in the historical KV store (line 6). Finally, we lock *undo* in the version chain and physically delete it (line 7).

5 TEMPORAL QUERY ENGINE

AeonG inherits and extends scan and expand operators to empower consistent and efficient temporal query processing.

Algorithm 1: Data migration

```
Function Migrate(CT):
    Input: CT, committed transaction no longer active;
    undo ← ∅; //unreclaimed version;
    kv ← ∅; //reclaimed version;
    foreach undo ∈ CT do
        kv=encode2KV(undo);
        KV_store::put(kv);
        physically delete undo;
End Function
```

5.1 Scan Operator

AeonG uses the scan operator to efficiently fetch vertex versions while ensuring data consistency for both current data and historical data. We elaborate on it from the aspect of fetching data from each storage component. When fetching data from the current storage, it is essential to ensure consistent data capture in the presence of concurrent transactions. To achieve this, we start by locating relevant vertex object(s) of interest. For each vertex object, we first employ the snapshot visibility check [33] to find a visible version of the given transaction. All versions preceding this visible version in the version chain are candidate legal versions we may want. Then we utilize a legal check mechanism, which verifies whether each candidate version v' is legal to the given query time condition, as per the following equation.

$$\omega.st \leq C.t_2 \wedge \omega.ed > C.t_1 \tag{2}$$

Here, $\omega.st$ and $\omega.ed$ represent the start and end time of v''s lifespan, respectively; C represents the time condition of the given query with begin time t_1 and end time t_2. For a time-point query, $t_1 = t_2$.

When fetching data from the historical storage, there is no need to handle transaction conflicts as the historical storage serves read-only queries that users cannot change the data in the historical storage. Therefore, we directly employ the legal check mechanism to get desired versions. To further enhance digging out historical versions, we employ an anchor-based skip retrieval strategy to reconstruct desired versions. To restore a specific legal version v', we directly seek the most recent anchor v in the KV store by the probe prefix "AV:id:C", where 'AV' represents the anchors in the VP segment, id is the unique id of interest vertex and C is the given query time constraint. We then assemble v' with all previous versions from v to v'. Thanks to the special design of the key-value format in the historical storage, we can leverage the probe prefix to swiftly find the nearest anchor.

Algorithm 2 shows how to fetch vertices from the hybrid storage engine. We start by scanning from the vertex object v, which is either the first vertex of the whole graph or the vertex pointed by the index (line 2). We first retrieve data from the current storage (lines 5-8). We check whether v and its historical unreclaimed versions are visible to the current transaction (line 6) and whether they are legal using the function `TemporalCheck()` based on Equation 1 (line 7). Next, we catch data from the historical storage (line 10) using the function `FetchFromKV()`. We first find the most recent anchor kv_a based on the probe prefix (line 15). We then seek all previous deltas kv_d that satisfy temporal check (line 19) and assemble kv_a with them to get desired versions (line 18). For example, consider retrieving v_2 at time t_n in Figure 4. We first locate the current version

Algorithm 2: Retrieving vertices

```
1  Function VertexRead(C):
       Input: C, temporal condition;
       Output: Σ, the result set;
2      v ← the vertex which we start to scan;
3      while v do
4          // fetch from the current storage;
5          foreach v' ∈ (v ∪ v.versions) do
6              if !SnapshotCheck(v') then continue;
7              if TemporalCheck(v'.ω, C) then
8                  Σ ← Σ ∪ Reconstruct(v');
9          // fetch from the historical storage;
10         FetchFromKV(v.id, C, Σ);
11         v ← v.next();
12     return Σ;
13 Function FetchFromKV(id, C, Σ):
14     kv_a ← KV_store::seeknext(id, C);
15     kv_d ← KV_store::seeknext(id, kv_a.key);
16     while kv_d ∧ kv_d.ω.st ≤ C.t_2 do
17         kv_a ← combine (kv_a, kv_d);
18         if TemporalCheck(kv_d.ω, C) then
19             Σ ← Σ ∪ kv_a;
20         kv_d ← kv_d.next();
```

$v_{2.8}$ in the current storage (line 2). If $v_{2.8}$ fails `TemporalCheck()`, we then search for v_2's historical versions within the current storage's version chain (lines 5-8). If we fail to get the required version from this version chain, we finally fetch it from the historical storage: We seek the nearest anchor and collect all relevant deltas to reconstruct the legal version $v_{2.7}$ of interest (lines 13-20).

Complexity analysis. The scan operator queries versions of a vertex v in a dataset with a total of n vertices. This process consists of two parts: (1) locating the current version of v and (2) querying the historical versions of v. The complexity of locating the current version depends on the specific retrieval mechanisms selected in the current storage, such as $log(n)$ for B$^+$-tree index look-up and n for non-index lookup, denoted as $O(\iota(n))$. The complexity of querying the historical versions depends on the chosen approach for introducing temporal features. In AeonG, we first locate the nearest anchor. Since we organize historical versions in the key-value store using SkipList, the complexity of this process is $O(log(A_v))$, where A_v is the average number of anchors for vertices. Then, we sequentially scan deltas from the anchor until satisfying the query time condition, with a time complexity of $O(u)$, where u represents the average length of u_o defined in Equation 1. In conclusion, the scan operator has a complexity of $O(\iota(n) + log(A_v) + u)$.

5.2 Expand Operator

AeonG utilizes the expand operator to fetch linked edge and adjacency vertex versions. The overall design insight of the expand operator is similar to that of the scan operator, which employs different retrieval strategies in two separate storage engines. Additionally, the expand operator considers the retrieval of graph structures. We next elaborate on how the expand operator fetches edge and neighboring vertex versions of a given vertex.

Algorithm 3: Expanding Vertices

```
1 Function ExpandVertices (v, C):
    Input: v, the graph vertex need to expand; C, temporal
           condition;
    Output: Σ, the result set;
2   Σ_ve ← VERead(v, C) //get adjacency lists;
3   foreach (e_id, nv_id) ∈ Σ_ve do
4       Σ_e ← EdgeRead(e_id, f(C, v.ω)) ;
5       foreach e ∈ Σ_e do
6           Σ_nv ← VertexRead(nv_id, f(C, e.ω));
7           foreach nv ∈ Σ_nv do
8               Σ ← Σ ∪ (e, nv) ;
9   return Σ;
```

As shown in Algorithm 3, given a vertex v, we first use the function `VERead()` to access the v's adjacency list version(s) from the VE part/segment (line 2), which contains a list of (edge id e_{id}, neighbor vertex id nv_{id}) pairs. The function `VERead()` shares a similar logic as the function `VertexRead()` in Algorithm 2. We then fetch legal versions of specific semantic information of edges and adjacency vertices based on their unique ids (lines 3-8). We first obtain linked edge versions using the `EdgeRead()` function, which follows a similar logic to `VertexRead()` (line 4). To expedite the search for the linked edge version, we optimize the skip look-up strategy for anchor locating. Guided by Constraint 1 defined in Section 2.1, the edge must be legal for its connected vertices. This implies that the lifespan of the edge version must intersect with the lifespan of its connected vertex version. Since we already hold a scanned vertex version v, we can leverage v's lifespan ω to refine the probe time scope C based on the following equation.

$$f(C, \omega) = [max(C.t_1, \omega.st), min(C.t_2, \omega.ed)] \tag{3}$$

By so doing, we bypass more unnecessary versions, thus boosting the query performance. Next, we retrieve the neighbor vertex versions of each holding edge version based on nv_{id} (lines 5-6) with similar logic and get the final results (lines 7-8). For instance, consider continue to retrieve $v_{2.7}$'s linked edge and adjacency vertex at time t_n in Figure 4. We first obtain the corresponding version of $v_{2.7}$'s adjacency list, which contains $[(1, 0)]$ (line 2). Then, we fetch the corresponding edge version $e_{1.2}$ (line 4) and the neighboring vertex version $v_{0.1}$ (line 6).

Complexity analysis. The expand operator retrieves linked edges and adjacent vertices of a specific vertex version in the following three steps. First, we fetch the adjacency list version. Similar to the complexity of the scan operator, the associated complexity is $O(log(A_{ve}) + u)$, where A_{ve} is the total number of anchors for adjacency lists, and u is the average length of u_o. Next, for each pair (e_{id}, nv_{id}) in the adjacency list, we fetch the corresponding edge version. Finally, we locate the neighbor vertex version. Similar to the first step, the complexities of these two steps are $O(log(A_e) + u)$ and $O(log(A_v) + u)$, where A_{ve} and A_v are the total number of anchors for edges and vertices, respectively. In conclusion, the overall complexity of the expand operator is $O(log(A_{ve}) + u + D \times (log(A_e) + u + log(A_v) + u))$, where D is the average number of vertex degrees. We would like to highlight that the complexity of both Algorithm 2 and Algorithm 3 is at the $O(log(n))$ level, which is reasonable and generally acceptable in graph query processing [28, 41, 42].

5.3 Proof of the Correctness

We now prove that we ensure the transaction to fetch consistent graph data from two isolated storage engines.

Theorem 1. *Given a read-only transaction T fetching temporal graph data with a condition time C, T can be verified to have retrieved the data from the hybrid storage with a consistent state.*

Proof. First, temporal graph data are globally consistent. The lifespan ω of each version is the commit time of the transaction, allocated by the system from a global clock. As a result, the timeline of two isolated storages follows the same serialization order. Second, we employ a fetching strategy to return consistent requested data based on the lifespan ω. For a version o in the current storage, we verify its visibility to T and then align o's ω to C (lines 6-8, Algorithm 2). For o in the historical storage, we solely align o's ω to C (line 10, Algorithm 2). By so doing, we can guarantee data consistency within each separate storage. It is possible to find a version o duplicated in both the current storage and the historical storage. Unreclaimed versions in the current storage might be transferred to the historical storage before being cut from the version chain during the asynchronous migration process (Algorithm 1). However, this does not lead to errors, as both versions share the same lifespan ω and contain identical information. In such cases, we incorporate a deduplication step, retaining only the version from the current storage. Considering all data-fetching scenarios, we ensure overall data consistency.

6 IMPLEMENTATION

AeonG is built on Memgraph[4] and RocksDB[40]. Memgraph is a commercial native graph database that supports the property graph model, Cypher, and MVCC. We utilize and extend Memgraph to serve as the primary database engine, providing the basic query engine and current storage engine for AeonG. We then integrate RocksDB, a popular KV store, into Memgraph as the historical storage to manage historical data. We would like to emphasize that our proposed temporal graph management mechanism is generally applicable to native graph databases that support MVCC.

Query engine. AeonG extends the *parser* and *executor* components of the query engine in Memgraph to support temporal queries. AeonG extends the *parser* to recognize temporal queries defined in Section 2.2, incorporating the temporal qualifier into `Cypher.g4` and enhancing `CypherMainVisitor()` to recognize temporal qualifier. Furthermore, AeonG enhances the *executor* by modifying two fundamental operators: scan and expand operators. In the `ScanAllCursor.Pull()` function, besides retrieving current vertices, we introduce a function `AddHistoricalVertices()` to capture both unreclaimed and reclaimed historical versions (Algorithm 2). In the `ExpandCursor.Pull()` function, a similar adaptation is made with the inclusion of the function `AddHistoricalEdges()` for getting historical edges and neighbor vertices (Algorithm 3).

Storage engine. The storage engine of AeonG is hybrid, consisting of a *current* storage and a *historical* storage, as detailed in Section 4. The *current* storage is derived from Memgraph's storage,

Workload Characteristics

	T-mgBench	T-LDBC	T-gMark			
			Bib	WD	LSN	SP
# of Vertices	10K	3,181K	100K	103K	100K	100K
# of Edges	122K	17,256K	121K	93K	200K	385K
Density	12.17	5.42	1.2	0.90	2	3.85
# of Vertex Labels	1	8	5	24	15	7
# of Edge Labels	1	25	4	82	27	7
# of Graph Operations for Data Generation	320K	1M	320K	320K	320K	320K

where the `Vertex` structure maps to the vertex object, the `EdgeRef` structure corresponds to the edge object, and the `Delta` structure represents historical unreclaimed versions. We then associate the time dimension with those structures to introduce temporal support, as discussed in Section 4.1. Timestamps are assigned by a global clock when relevant transactions are committed, with a time granularity of milliseconds. AeonG integrates RocksDB into Memgraph as the *historical* storage by starting a RocksDB process when the Memgraph instance starts. We introduce a function, `Migrate()`, within Memgraph's `CollectGarbage()` function to transfer unreclaimed data to a key-value pair and subsequently migrate them to RocksDB (Algorithm 1). We further implement a distributed version of AeonG, named AeonG-D, using TiKV [43], an efficient distributed key-value store, for historical storage by replacing the interfaces of RocksDB with TiKV. We introduce a system parameter, `retention_period`, in AeonG to set the historical data retention period. For instance, setting `retention_period` to one month enables the periodic removal of historical graph data generated one month ago from the historical storage.

7 EVALUATION

In this section, we first introduce the experimental setup. We then compare AeonG against two state-of-the-art temporal systems, Clock-G and T-GQL, and provide in-depth performance analyses for AeonG, with two metrics: 1) latency of temporal queries/graph operations; 2) storage overheads of temporal graph data.

7.1 Experimental Setup

AeonG is built on Memgraph $v2.2.0$, RocksDB $v6.14.6$, and TiKV $v7.1.2$ for evaluation. We run the experiments in a cluster of up to 5 nodes. Each node is equipped with 32 Intel(R) Xeon(R) Gold 5220 CPU @ 2.20GHz, 128 GB memory, running CentOS 7.9.

7.1.1 Baseline Systems. We compare AeonG with two baseline systems that support temporal features:

T-GQL [7]: A state-of-the-art graph database that assigns a time period to each graph object (vertex or edge) for temporal support. Since we cannot obtain the source code of T-GQL, for fair comparisons, we implement T-GQL based on Memgraph. Note that T-GQL stores all the vertices and edges in memory.

Clock-G [19]: A state-of-the-art graph storage engine that manages temporal data by periodically creating snapshots of the entire databases. Since Clock-G is not open-sourced, we implement its temporal data management approach into our codebase for fair comparisons. We record both the snapshots and the logs between successive snapshots in RocksDB. We further introduce a query engine like that used in AeonG to support temporal queries. By default, Clock-G creates a snapshot after executing 80k graph operations.

Table 1: Temporal Queries of T-mgBench

Query	Statement
Q1	Match (n: User {id: *$id*}) FOR TT AS OF t RETURN n
Q2	Match (n: User {id: *$id*}) FOR TT From t_1 to t_2 RETURN n
Q3	Match (n: User {id: *$id*})-[e]->(m) FOR TT AS OF t RETURN n,e,m
Q4	Match (n: User {id: *$id*})-[e]->(m) FOR TT From t_1 to t_2 RETURN n,e,m

7.1.2 Workloads. We conduct the experiments using three temporal-enhanced workloads, characteristics of which are detailed in Table **??**. We next describe each of these three workloads.

T-mgBench is based on the real-world Pokec dataset [44], which is used in Memgraph's mgBench [45] workload. As outlined in Table 1, T-mgBench includes four temporal queries, by extending the non-temporal queries in mgBench with temporal dimension. Specifically, we add "FOR TT AS OF t" to Q1 and Q3, forming "time-point" queries, and add "FOR TT FROM t_1 to t_2" to Q2 and Q4, forming "time-slice" queries.

T-LDBC derives from LDBC [46], a well-known synthetic graph workload, by incorporating "FOR TT AS OF t" into the LDBC IS queries (IS1-IS7). More specifics of these queries can be found in Table I in our Appendix.

T-gMark is based on gMark[47], a well-known synthetic graph workload. It consists of four datasets as shown in Tables **??**. We use gMark's query generation tool to create non-temporal queries, and then transform them into temporal queries by adding the time condition "FOR TT AS OF t". The query generation follows gMark's default configuration, which includes constraints on arity (0-4), query shape (25% chain, 25% star-chain, 25% cycle, and 25% star), selectivity (33% constant, 33% linear, and 33% quadratic), probability recursion (50%), and query size ([1,1], [3,4], [1,3], [2,4]).

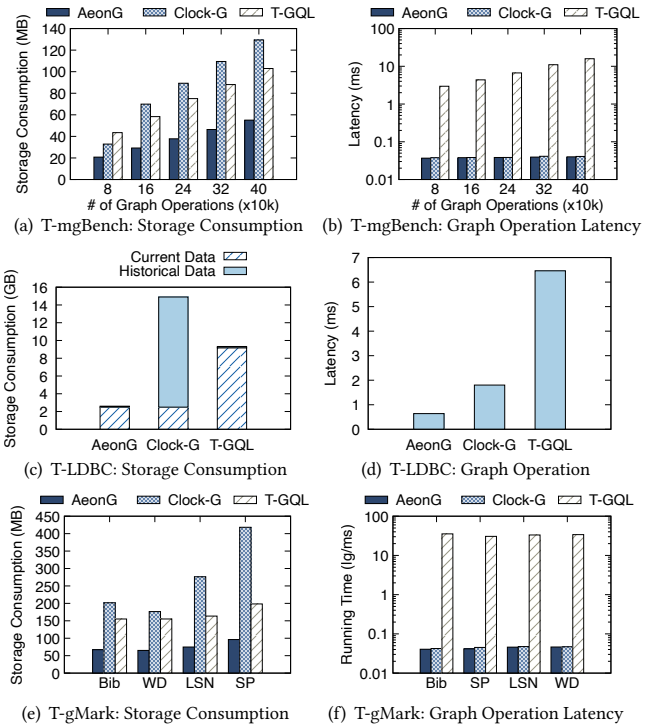
To effectively evaluate the efficiency of temporal features in AeonG and baseline systems, for each workload, we generate additional historical data before evaluations. Unless otherwise specified, we first use the data generation tools from the original workload to create the initial dataset, and then execute graph operations with a mix of 80% updates, 10% creates, and 10% deletes to generate historical data. The access distribution of update operations and queries follows the Zipf [48] distribution to simulate real-world graph manipulation scenarios.

7.1.3 Default Configuration. By default, the Zipf distribution factor is set to 1.1. The parameters of the adaptive anchoring approach defined in Equation 1 are configured as $\tau_1 = 1k$, $\tau_2 = 10k$ and $c = 1\%$. The `retention_period` discussed in Section 6 is set to 0, indicating that all historical data will be retained permanently.

7.2 AeonG vs Baseline Systems

We now compare AeonG with two baseline systems, Clock-G and T-GQL. As T-GQL is an in-memory database, to ensure fair comparisons, we make sure all data is cached in memory for AeonG and Clock-G by configuring RocksDB's MemTable size to 640MB.

7.2.1 Experiment on the storage consumption. We first conduct experiments using T-mgBench, and plot the storage consumption of each system under different numbers of graph operations in Figure 8(a). As observed, AeonG reduces the storage consumption by up to 2.4× compared to Clock-G and 2.09× compared to T-GQL, with



(a) T-mgBench: Storage Consumption (b) T-mgBench: Graph Operation Latency

(c) T-LDBC: Storage Consumption (d) T-LDBC: Graph Operation

(e) T-gMark: Storage Consumption (f) T-gMark: Graph Operation Latency

Figure 8: Comparisons on Storage Consumption and Graph Operation Latency

the increased number of graph operations. The lowest storage consumption of AeonG can be attributed to our "anchor+delta" strategy, which compactly stores most historical graph data as deltas, minimizing the storage consumption for maintaining temporal data. In contrast, T-GQL's graph model prevents it from storing historical graph data compactly, while Clock-G periodically creates historical snapshots of the entire graph. Both systems incur higher storage overheads than AeonG.

We also run experiments using T-LDBC and T-gMrak to evaluate the storage consumption of these systems. As shown in Figure 8(c), AeonG exhibits up to 5.73× and 3.59× lower storage consumption compared to Clock-G and T-GQL, under T-LDBC. Further, as observed in Figure 8(e), the storage consumption of AeonG is lower than that of Clock-G and T-GQL by up to a 4.34× and a 2.39×, under T-gMrak. This trend is consistent with that observed in Figure 8(a), demonstrating that AeonG still achieves lower storage overhead when handling large and complex graph workloads.

7.2.2 Experiments on the graph operation latency. We then evaluate the graph operation latency with varying numbers of graph operations using T-mgBench. As shown in Figure 8(b), AeonG performs similarly to Clock-G, but significantly outperforms T-GQL by up to 397.06×. As the number of graph operations grows, both AeonG and Clock-G exhibit a performance degradation of 5.4× from 80k to 400k, whereas T-GQL shows a much larger degradation of 34.95×. The performance difference is mainly due to that T-GQL does not separate the storage of current and historical data. Therefore, graph operations, such as updates, require traversing through a larger number of graph objects (both current and historical data) to reach the specific graph object for updating. However, AeonG separate current and historical data, leading to much smaller latency overheads. The similar performance of AeonG and Clock-G in T-mgBench can be explained as the overhead from snapshot creation is not significant when handling a relatively small size graph. Therefore, to further evaluate storage operation latency with larger graphs, we conducted additional experiments using the T-LDBC workload. As observed in Figure 8(d), the graph operation latency of AeonG is lower than Clock-G and T-GQL by up to 2.82× and 10.11×, respectively. As T-LDBC is more substantial, Clock-G requires extra CPU and IO resources to periodically create large historical snapshots, which can negatively affect graph operation performance due to resource contention. As shown in Figure 8(f), AeonG performs similarly to Clock-G and significantly outperforms T-GQL by up to 766×. This trend is consistent with the one observed in Figure 8(b).

7.2.3 Experiments on temporal query latency. We now analyze the performance of temporal queries under various configurations. We first conduct performance evaluations across various temporal queries in T-mgBench, and plot the query latency on different temporal queries in Figure 9(a). By default, we set the time slice length for Q2 and Q4 to 100s. We can observe that AeonG reduces the query latency by 2.57× compared to Clock-G and 37.57× compared to T-GQL. The superior performance of AeonG can be attributed to our built-in query engine, which employs an efficient anchor-based version retrieval technique to avoid unnecessary version traversal. In contrast, to access desired historical graph elements, T-GQL necessitates traversing the entire graph , while Clock-G requires fetching the corresponding historical snapshot and appending logs on it, thereby resulting in slower performance.

We then study the latency of temporal query Q1 with varying the number of graph operations. Figure 9(b) shows that AeonG outperforms Clock-G by up to 1.43× and achieves an up to 47.18× improvement compared to T-GQL. This performance gap becomes increasingly pronounced with a growing number of graph operations. As discussed, AeonG exhibits better temporal query performance due to the proposed query engine specifically optimized for temporal query processing. We further evaluate the query latency of Q3 across "cold", "warm", and "hot" queries. We categorize these queries based on the vertices they access. For example, a query accessing "hot" vertices is classified as a "hot" query. We divide all vertices in the database into "cold", "warm", and "hot" categories according to their access possibility, which ranges from 0% to 100% based on the Zipf distribution. As shown in Figure 9(c), AeonG outperforms the next-best system, Clock-G, in all query categories by up to 2.21×. We can also observe that "hot" queries, which require accessing more historical data, generally have lower performance than "warm" and "cold" queries. However, in AeonG, "hot" queries are only 1.29× slower than "warm" queries, while in T-GQL and Clock-G, "hot" queries underperform "warm" queries by up to 2.14× and 3.77×, respectively. As discussed, the smaller performance gap of AeonG can be attributed to our anchor-based version retrieval technique, which avoids unnecessary version traversal. We also study the latency of temporal query Q4 with varying time slice length from 1s to 200s. As observed in Figure 9(d), AeonG outperforms Clock-G and T-GQL by up to 2.27× and 33.23×, respectively,

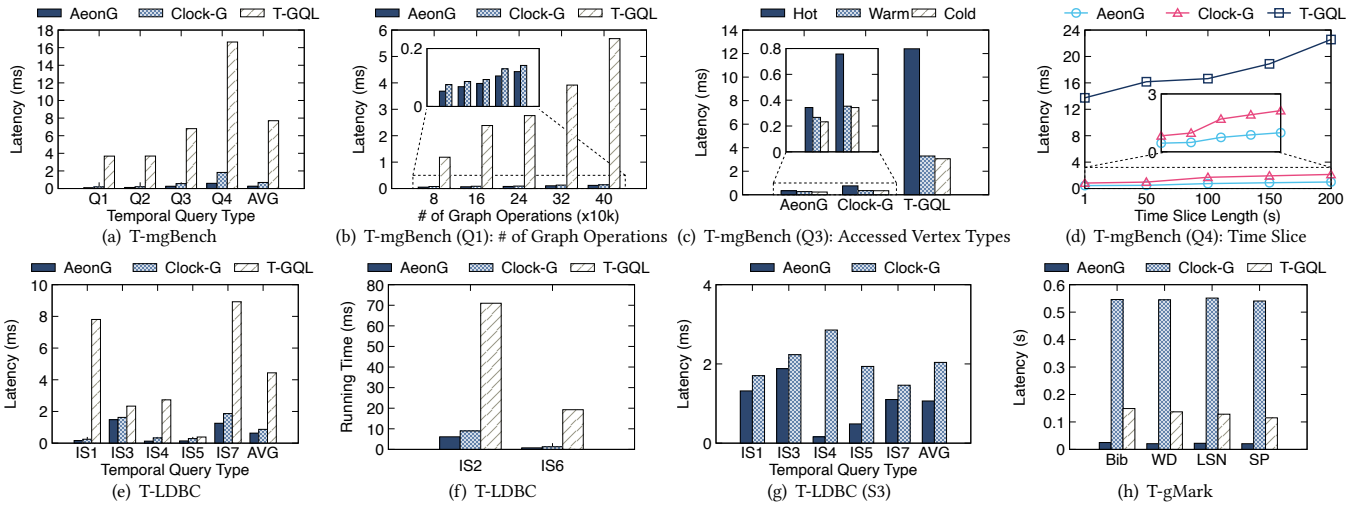


Figure 9: Comparisons on Temporal Query Latency

showing a consistently superior performance under different time slice lengths.

We additionally evaluate the temporal query performance on T-LDBC. As depicted in Figure 9(e) and 9(f), AeonG outperforms among all temporal query types and achieves lower latency by up to 1.46× and 12.07× than Clock-G and T-GQL, in alignment with the trends observed in Figure 9(a). We also rerun the experiments on T-LDBC by utilizing a larger graph with 9.28M vertices and 52.7M edges. During these experiments, T-GQL encounters the out-of-memory issue due to the increased graph size. However, as shown in Figure 9(g), AeonG continues to outperform Clock-G by up to 2.06×, which aligns with the findings shown in Figure 9(e). Furthermore, we conduct experiments using T-gMrak. As depicted in Figure 9(h), AeonG consistency outperforms in all the datasets and demonstrates up to 26.16× faster temporal query performance than Clock-G and 6.56× faster than T-GQL.

7.3 Performance Analysis on AeonG

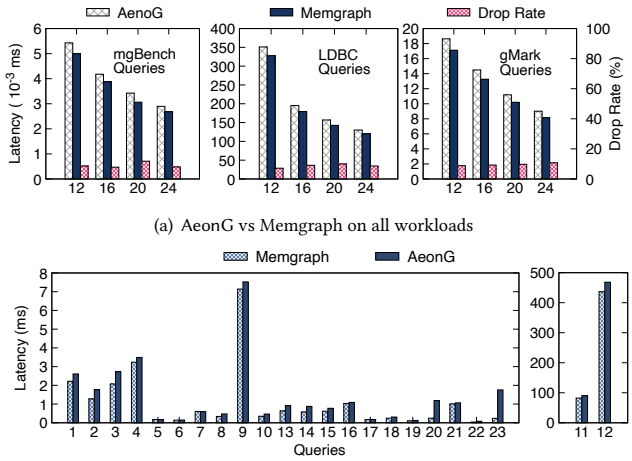
We now provide an in-depth analysis of AeonG's performance under diverse configurations. In the following experiments, we fix RocksDB's MemTable size to the default value of 64MB.

7.3.1 The performance of non-temporal queries. We first analyze the performance of AeonG on non-temporal queries to study the impact of introducing temporal features in its fundamental system, Memgraph. We use various non-temporal queries defined in three origin unextended workloads: mgBench, LDBC, and gMark. We run corresponding queries based on the datasets generated by T-mgBench, T-LDBC, and T-gMark, and plot the average query latency of each workload in Figure 10. The results indicate that AeonG experiences an acceptable performance drop of up to 9.74% compared with Memgraph, a trade-off for its support of temporal queries. In addition, we provide a detailed performance breakdown for each specific query of mgBench in Figure 10(b). We can observe that AeonG maintains comparable performance with Memgraph in most of the tested queries. AeonG adopts a design that separates the current database and asynchronously transfers historical data, ensuring minimal impact on dominant non-temporal queries.

7.3.2 The impact of historical data migration. We next use the T-mgBench workload to study the query performance with varying the GC interval to control the frequency of historical data migration. We plot the latency of queries across different data types: current, reclaimed, and unreclaimed data, and graph operation in Figure 11(a). As observed, the query performance for current and unreclaimed data is relatively similar, both outperforming reclaimed data queries by up to 25.8%. This difference is attributed to the fact that querying current and unreclaimed data both need to traverse the version chain in the current storage, while querying reclaimed data requires the additional step of reconstructing a historical version using anchors and deltas in the historical storage, as detailed in Section 4. Further, we note that increasing the GC interval from 1s to 1000s leads to a 17.3% decrease in the graph operation latency and a 9.5% increase in the query latency. This is expected as less frequent migrations can reduce contention with graph operation, thereby enhancing graph operation performance. In contrast, less frequent migrations result in longer version-chain traversal in the current storage, negatively impacting query performance.

7.3.3 The analysis on the anchor interval. We evaluate the effectiveness of our adaptive anchoring approach using the T-LDBC workload. As shown in Figure 11(b), when we assign a fixed anchor interval u to each graph object and vary u from 1 to 1000, we observe that storage consumption of the historical storage decreases by 2.69× and the temporal query latency increases by 2.15×. In contrast, our adaptive anchoring approach consistently achieves near-optimal query performance and storage efficiency against all fixed anchor interval settings, because of its ability to properly balance query latency and storage overhead efficiency.

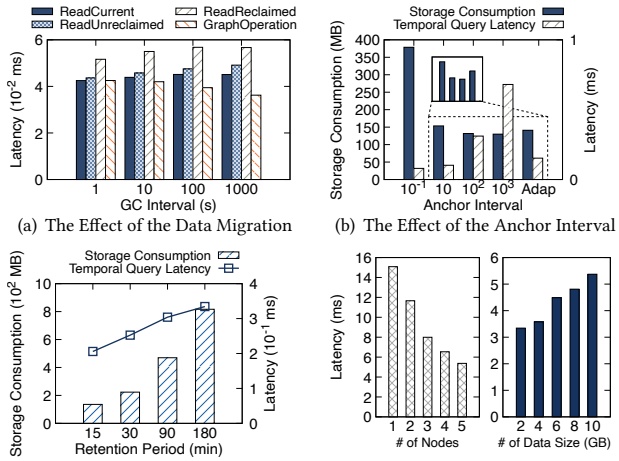
7.3.4 The impact of the historical retention period. We utilize the T-LDBC workload to evaluate the historical storage overhead and temporal query latency with varying historical data retention periods. We simulate one day's amount of graph evolution in just one



(a) AeonG vs Memgraph on all workloads

(b) Running time of specific queries in mgBench

Figure 10: AeonG vs Memgraph on non-temporal queries



(a) The Effect of the Data Migration

(b) The Effect of the Anchor Interval

(c) Varying the Retention Period

(d) Extend on TiKV

Figure 11: Performance Breakdown Analysis on AeonG

minute, which is done by assuming a daily operation count of 100k and executing these operations within one minute. Consequently, we set historical data retention periods at 15, 30, 90, and 180 minutes, simulating real-world scenarios of half a month, one month, one quarter, and half a year, respectively. The results, shown in Figure 11(c), demonstrate that the storage consumption increases by 6.02× and query performance decreases by 1.62× as the data retention period extends from 15 to 180 minutes. This trend is expected since longer retention period results in more historical data being maintained, leading to decreased query performance. Based on this observation, we consider enabling users to set a proper `retention_period` to achieve a balance among storage overhead, historical data duration, and query performance.

7.3.5 The scalability of AeonG-D. We now deploy AeonG-D and TiKV across 5 nodes by default, with historical data horizontally partitioned among these nodes. First, we evaluate the impact of increasing the server count on performance with 10GB data volume with the T-LDBC workload. The results, shown in the left part of Figure 11(d), indicate that the temporal query of AeonG-D decreases by up to 2.8× when scaling from 1 to 5 servers. The scalability of AeonG-D can be attributed to the improved parallelism achieved by adding more servers, where each server can independently process historical data retrieval requests with its TiKV instance. Second, we assess temporal query latency using T-LDBC with the data volume increasing from 2GB to 10GB. As shown in the right part of Figure 11(d), the latency of AeonG-D increases by up to 1.6×, which is expected due to the greater cost of fetching graph objects from a larger database. Similar trends are also reported in [19, 23, 41].

8 RELATED WORK

Temporal data management has reached a mature stage in the field of relational database systems [49–55]. Nevertheless, the temporal data management in the graph context remains relatively limited. In this section, we provide a review of temporal graph data management systems through two primary approaches: one at the application level and the other at the internal system level.

One approach to supporting temporal dimension is to utilize commercial graph databases by attaching temporal metadata. Cattuto et al. [8] propose a temporal model where the temporal data is represented as frames, with a frame being defined as the finest unit of temporal aggregation. Debrouvier et al. enhance the property graph model to store the temporal graph [7]. They assume that Objects, Attributes, and Values are stored as conventional property graph vertices, whereas time intervals are stored as properties of these vertices. Temporal edges are, in turn, stored as conventional edges, with time intervals as one of their properties. Other works exploit the graph database by representing the temporal dimension as an interval attribute to graph objects [9–13]. These systems are implemented at the application level upon existing database systems. They may exhibit unpredictable performance, as the underlying storage and query engines are designed for static graphs rather than evolving ones.

Another line of research focuses on underlying storage engines to organize temporal graphs. Their primary goal is to handle growing historical data while enabling efficient querying. In this regard, two storage approaches, *Copy* and *Log*, are widely used to manage temporal graph data [14–23]. The *Copy* approach [14, 15] stores a complete graph state whenever a batch of updates occurs. While this simplifies graph querying, it results in excessive redundancy in the stored graph information. In contrast, the *Log* approach [16, 17] records every graph update activity in a log. This approach is more compact but requires expensive reconstruction when executing a temporal graph query. As a trade-off between query performance and space overhead, the *Copy+Log* approach [18–23] combines a finite set of snapshots with a list of deltas between them. However, we argue that *Copy+Log* is a sub-optimal approach since it still necessitates significant storage overhead to materialize the entire graph. Furthermore, all the above approaches primarily concentrate on offline historical data, limiting their usability for online queries. Worse still, to the best of our knowledge, they lack support for either a powerful temporal graph data model or a declarative temporal query language, restricting the convenience for users.

9 CONCLUSION

In this paper, we present AeonG, a new graph database that efficiently offers built-in temporal support. AeonG is based on a novel temporal property graph model. To fit this model, we design a temporal-enhanced storage engine and a query engine. We first provide a hybrid storage engine with an asynchronous migration, guaranteeing small performance degradation on current storage and optimizing storage efficiency for historical storage. We then introduce a temporal query engine, which employs an legal examination technique to ensure data consistency and an anchor-based version retrieval technique to boost query performance. We conducted extensive experimental evaluations using three workloads. The results demonstrated that AeonG achieves up to 5.73× lower storage overhead, 2.82× lower latency for graph operations, and 2.57× lower latency for temporal queries against the next-best system, while introducing only 9.74% performance degradation for supporting temporal features.

REFERENCES

[1] "Neo4j." https://neo4j.com.
[2] "Arangodb." https://www.arangodb.com.
[3] "Dgraph." https://dgraph.io.
[4] "Memgraph." https://memgraph.com.
[5] A. Abdallah, M. A. Maarof, and A. Zainal, "Fraud detection system: A survey," *J. Netw. Comput. Appl.*, vol. 68, pp. 90–113, 2016.
[6] W. Laddada and C. Ray, "Graph-based analysis of maritime patterns of life," in *Proceedings of the GAST Workshop, 20th Journées Francophones Extraction et Gestion des Connaissances (EGC)*, pp. 1–14, 2020.
[7] A. Debrouvier, E. Parodi, M. Perazzo, V. Soliani, and A. A. Vaisman, "A model and query language for temporal graph databases," *VLDB J.*, vol. 30, no. 5, pp. 825–858, 2021.
[8] C. Cattuto, M. Quaggiotto, A. Panisson, and A. Averbuch, "Time-varying social networks in a graph database: a neo4j use case," in *GRADES*, p. 11, CWI/ACM, 2013.
[9] Z. Liu, C. Wang, and Y. Chen, "Keyword search on temporal graphs," in *ICDE*, pp. 1807–1808, IEEE Computer Society, 2018.
[10] C. Rost, K. Gómez, M. Täschner, P. Fritzsche, L. Schons, L. Christ, T. Adameit, M. Junghanns, and E. Rahm, "Distributed temporal graph analytics with GRADOOP," *VLDB J.*, vol. 31, no. 2, pp. 375–401, 2022.
[11] M. Yu, "A graph-based spatiotemporal data framework for 4d natural phenomena representation and quantification-an example of dust events," *ISPRS Int. J. Geo Inf.*, vol. 9, no. 2, p. 127, 2020.
[12] L. Zheng, L. Zhou, Z. Xin, L. Li, and W. Liu, "The spatio-temporal data modeling and application based on graph database," in *2017 4th International Conference on Information Science and Control Engineering (ICISCE)*, 2017.
[13] G. C. Durand, M. Pinnecke, D. Broneske, and G. Saake, "Backlogs and interval timestamps: Building blocks for supporting temporal queries in graph databases," in *EDBT/ICDT Workshops*, vol. 1810 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2017.
[14] U. Khurana and A. Deshpande, "Efficient snapshot retrieval over historical graph data," in *ICDE*, pp. 997–1008, IEEE Computer Society, 2013.
[15] J. Byun, S. Woo, and D. Kim, "Chronograph: Enabling temporal graph traversals for efficient information diffusion analysis over time," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 3, pp. 424–437, 2020.
[16] B. A. Steer, F. Cuadrado, and R. G. Clegg, "Raphtory: Streaming analysis of distributed temporal graphs," *Future Gener. Comput. Syst.*, vol. 102, pp. 453–464, 2020.
[17] P. Macko, V. J. Marathe, D. W. Margo, and M. I. Seltzer, "LLAMA: efficient graph analytics using large multiversioned arrays," in *ICDE*, pp. 363–374, IEEE Computer Society, 2015.
[18] Y. Miao, W. Han, K. Li, M. Wu, F. Yang, L. Zhou, V. Prabhakaran, E. Chen, and W. Chen, "Immortalgraph: A system for storage and analysis of temporal graphs," *ACM Trans. Storage*, vol. 11, no. 3, pp. 14:1–14:34, 2015.
[19] M. Massri, Z. Miklós, P. R. Parvédy, and P. Meye, "Clock-g: A temporal graph management system with space-efficient storage technique," in *ICDE*, pp. 2263–2276, IEEE, 2022.
[20] P. Kumar and H. H. Huang, "Graphone: A data store for real-time analytics on evolving graphs," *ACM Trans. Storage*, vol. 15, no. 4, pp. 29:1–29:40, 2020.
[21] W. Han, Y. Miao, K. Li, M. Wu, F. Yang, L. Zhou, V. Prabhakaran, W. Chen, and E. Chen, "Chronos: a graph engine for temporal graph analysis," in *EuroSys*, pp. 1:1–1:14, ACM, 2014.
[22] W. Han, K. Li, S. Chen, and W. Chen, "Auxo: a temporal graph management system," *Big Data Min. Anal.*, vol. 2, no. 1, pp. 58–71, 2019.
[23] U. Khurana and A. Deshpande, "Storing and analyzing historical graph data at scale," in *EDBT*, pp. 65–76, OpenProceedings.org, 2016.
[24] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. L. Reutter, and D. Vrgoc, "Foundations of modern query languages for graph databases," *ACM Comput. Surv.*, vol. 50, no. 5, pp. 68:1–68:40, 2017.
[25] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An evolving query language for property graphs," in *SIGMOD Conference*, pp. 1433–1445, ACM, 2018.
[26] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, A. Green, J. Hidders, B. Li, L. Libkin, V. Marsault, W. Martens, F. Murlak, S. Plantikow, O. Savkovic, M. Schmidt, J. Sequeda, S. Staworko, D. Tomaszuk, H. Voigt, D. Vrgoc, M. Wu, and D. Zivkovic, "Pg-schema: Schemas for property graphs," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 198:1–198:25, 2023.
[27] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, K. W. Hare, J. Hidders, V. E. Lee, B. Li, L. Libkin, W. Martens, F. Murlak, J. Perryman, O. Savkovic, M. Schmidt, J. F. Sequeda, S. Staworko, and D. Tomaszuk, "Pg-keys: Keys for property graphs," in *SIGMOD Conference*, pp. 2423–2436, ACM, 2021.
[28] X. Zhu, M. Serafini, X. Ma, A. Aboulnaga, W. Chen, and G. Feng, "Livegraph: A transactional graph storage system with purely sequential adjacency list scans," *Proc. VLDB Endow.*, vol. 13, no. 7, pp. 1020–1034, 2020.
[29] A. Dubey, G. D. Hill, R. Escriva, and E. G. Sirer, "Weaver: A high-performance, transactional graph database based on refinable timestamps," 2015.
[30] H. Chen, C. Li, C. Zheng, C. Huang, J. Fang, J. Cheng, and J. Zhang, "G-tran: Making distributed graph transactions fast," 2021.
[31] Y. Wu, J. Arulraj, J. Lin, R. Xian, and A. Pavlo, "An empirical evaluation of in-memory multi-version concurrency control," *Proc. VLDB Endow.*, vol. 10, no. 7, pp. 781–792, 2017.
[32] K. G. Kulkarni and J. Michels, "Temporal features in SQL: 2011," *SIGMOD Rec.*, vol. 41, no. 3, pp. 34–43, 2012.
[33] T. Neumann, T. Mühlbauer, and A. Kemper, "Fast serializable multi-version concurrency control for main-memory database systems," in *SIGMOD Conference*, pp. 677–689, ACM, 2015.
[34] J. Kim, K. Kim, H. Cho, J. Yu, S. Kang, and H. Jung, "Rethink the scan in MVCC databases," in *SIGMOD Conference*, pp. 938–950, ACM, 2021.
[35] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A critique of ANSI SQL isolation levels," in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data, San Jose, California, USA, May 22-25, 1995* (M. J. Carey and D. A. Schneider, eds.), pp. 1–10, ACM Press, 1995.
[36] H. Li, Z. Zhao, Y. Cheng, W. Lu, X. Du, and A. Pan, "Efficient time-interval data extraction in mvcc-based rdbms," *World Wide Web*, 2018.
[37] D. Ports and K. Grittner, "Serializable snapshot isolation in postgresql," *Proceedings of the Vldb Endowment*, vol. 5, no. 12, pp. 1850–1861, 2012.
[38] P. Gupta, A. Mhedhbi, and S. Salihoglu, "Columnar storage and list-based processing for graph database management systems," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2491–2504, 2021.
[39] C. Kankanamge, S. Sahu, A. Mhedhbi, J. Chen, and S. Salihoglu, "Graphflow: An active graph database," in *SIGMOD Conference*, pp. 1695–1698, ACM, 2017.
[40] "Rocksdb." https://rocksdb.org/.
[41] A. G. Labouseur, J. Birnbaum, P. W. Olsen, S. R. Spillane, J. Vijayan, J.-H. Hwang, and W.-S. Han, "The g* graph database: efficiently managing large distributed dynamic graphs," *Distributed and Parallel Databases*, vol. 33, pp. 479–514, 2015.
[42] D. D. Leo and P. A. Boncz, "Teseo and the analysis of structural dynamic graphs," *Proc. VLDB Endow.*, vol. 14, no. 6, pp. 1053–1066, 2021.
[43] "Tikv.." https://tikv.org/.
[44] "Pokec." https://pokec.azet.sk/.
[45] "Mgbench." https://memgraph.com/benchgraph.
[46] A. Iosup, T. Hegeman, W. L. Ngai, S. Heldens, A. Prat-Pérez, T. Manhardt, H. Chafi, M. Capota, N. Sundaram, M. J. Anderson, I. G. Tanase, Y. Xia, L. Nai, and P. A. Boncz, "LDBC graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms," *Proc. VLDB Endow.*, vol. 9, no. 13, pp. 1317–1328, 2016.
[47] G. Bagan, A. Bonifati, R. Ciucanu, G. H. L. Fletcher, A. Lemay, and N. Advokaat, "gmark: Schema-driven generation of graphs and queries," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 4, pp. 856–869, 2017.
[48] G. K. Zipf, *Human Behaviour and the Principle of Least Effort: an Introduction to Human Ecology.* Addison-Wesley, 1949.
[49] A. Bolour, T. L. Anderson, L. J. Dekeyser, and H. K. T. Wong, "The role of time in information processing: a survey," *SIGART Newsl.*, vol. 80, pp. 28–46, 1982.
[50] R. T. Snodgrass and I. Ahn, "A taxonomy of time in databases," in *SIGMOD Conference*, pp. 236–246, ACM Press, 1985.
[51] G. Özsoyoglu and R. T. Snodgrass, "Temporal and real-time databases: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 7, no. 4, pp. 513–532, 1995.
[52] A. U. Tansel, J. Clifford, S. K. Gadia, S. Jajodia, A. Segev, and R. T. Snodgrass, eds., *Temporal Databases: Theory, Design, and Implementation.* Benjamin/Cummings, 1993.

[53] W. Lu, Z. Zhao, X. Wang, H. Li, Z. Zhang, Z. Shui, S. Ye, A. Pan, and X. Du, "A lightweight and efficient temporal database management system in TDSQL," *Proc. VLDB Endow.*, vol. 12, no. 12, pp. 2035–2046, 2019.

[54] Z. Zhao, W. Lu, H. Zhao, Z. He, H. Li, A. Pan, and X. Du, "T-SQL: A lightweight implementation to enable built-in temporal support in mvcc-based rdbmss," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 1028–1042, 2023.

[55] B. Salzberg and V. J. Tsotras, "Comparison of access methods for time-evolving data," *ACM Comput. Surv.*, vol. 31, no. 2, pp. 158–221, 1999.

Table 1: Temporal queries of T-LDBC workload

IS1	MATCH (n:Person {id:xx})-[:IS_LOCATED_IN]-(p:Place) **FOR TT AS OF** *t* RETURN n.firstName AS firstName, n.lastName AS lastName, n.birthday AS birthday, n.locationIP AS locationIp, n.browserUsed AS browserUsed, n.gender AS gender, n.creationDate AS creationDate, p.id AS cityId;
IS2	MATCH (:Person {id:xx})<-[:HAS_CREATOR]-(m)-[:REPLY_OF*0..]->(p:Post) MATCH (p)-[:HAS_CREATOR]->(c) **FOR TT AS OF** *t* RETURN m.id as messageId, CASE m.content is not null WHEN true THEN m.content, ELSE m.imageFile, END AS messageContent, m.creationDate AS messageCreationDate, p.id AS originalPostId, c.id AS originalPostAuthorId, c.firstName as originalPostAuthorFirstName, c.lastName as originalPostAuthorLastName ORDER BY messageCreationDate DESC LIMIT 10;
IS3	MATCH (n:Person {id:xx})-[r:KNOWS]-(friend) **FOR TT AS OF** *t* RETURN friend.id AS personId, friend.firstName AS firstName, friend.lastName AS lastName, r.creationDate AS friendshipCreationDate ORDER BY friendshipCreationDate DESC, toInteger(personId) ASC;
IS4	MATCH (m:Message {id:xx}) **FOR TT AS OF** *t* RETURN CASE exists(m.content) WHEN true THEN m.content ELSE m.imageFile END AS messageContent, m.creationDate as messageCreationDate;
IS5	MATCH (m:Message {id:xx})-[:HAS_CREATOR]->(p:Person) **FOR TT AS OF** *t* RETURN p.id AS personId, p.firstName AS firstName, p.lastName AS lastName;
IS6	MATCH (m:Message {id:xx})-[:REPLY_OF*0..]->(p:Post) <-[:CONTAINER_OF]-(f:Forum)-[:HAS_MODERATOR]->(mod:Person) **FOR TT AS OF** *t* RETURN f.id AS forumId, f.title AS forumTitle, mod.id AS moderatorId, mod.firstName AS moderatorFirstName, mod.lastName AS moderatorLastName;
IS7	MATCH (m:Message {id:xx})<-[:REPLY_OF]-(c:Comment)-[:HAS_CREATOR]->(p:Person) OPTIONAL MATCH (m)-[:HAS_CREATOR]->(a:Person)-[r:KNOWS]-(p) **FOR TT AS OF** *t* RETURN c.id AS commentId, c.content AS commentContent, c.creationDate AS commentCreationDate, p.id AS replyAuthorId, p.firstName AS replyAuthorFirstName, p.lastName AS replyAuthorLastName, CASE r WHEN null THEN false ELSE true END AS replyAuthorKnowsOriginalMessageAuthor ORDER BY commentCreationDate DESC, toInteger(replyAuthorId) ASC;

A TEMPORAL QUERIES OF T-LDBC

We list the temporal queries of the T-LDBC workload used in our experiment in Table 1. The values of the vertex id are sampled from the respective datasets on which the queries are evaluated.